智慧電表
異常用電偵測系統

基於非監督式機器學習的AIoT 電力異常分析

組員
4112056045 資工三 莊明達
4112056048 資工三 鄭仲軒
4112064213 電資三 戴煦

組別:1 · 課程:AIoT · 指導老師:陳煥老師 · 2026

大綱

第一章 💡

動機

問題背景與痛點說明

第二章 📚

相關研究

文獻回顧與方法比較

第三章 🏗️

系統設計

架構圖與規格說明

第四章 💻

實作細節

資料處理、模型、Dashboard

第五章 📈

實驗結果

效能指標與比較分析

第六章 🏁

結論

總結與未來展望

大綱

第一章

動機

問題背景與痛點說明

第二章

相關研究

文獻回顧與方法比較

第三章

系統設計

架構圖與規格說明

第四章

結論

總結與未來展望

大綱

第一章

動機
Motivation

問題背景與痛點說明

第二章

相關研究
Related Work

文獻回顧與方法比較

第三章

系統設計
Proposed Design

架構圖與規格說明

我們想解決的問題:異常用電中的「竊電」偵測

- 竊電佔電力非技術性損失 (Non-Technical Lost, NTL) 超過九成
- Non-Technical Lost 造成的負面影響[1]:
  1. 電力用戶需要付更多電費
  2. 浪費高額的政府補貼
  3. 造成公共安全疑慮(如透過危險的連接方式竊電)
- 智慧電表雖普及, 但無「分析異常的功能」

[1] https://www.prnewswire.com/news-releases/96-billion-is-lost-every-year-to-electricity-theft-300453411.html

第一章　動機 Motivation

動機 | 相關研究 | 系統設計 | 結論

~$101.2B

根據Northeast Group研究
全球一年非技術性損失金額
（2021年，138個國家）[1]

~數日至月

人工抄表、檢查資料之滯後性

~423萬具

台灣安裝智慧電表數 [2]
人工無法處理海量數據

[1] https://northeast-group.com/2021/10/20/electricity-theft-non-technical-losses/
[2] https://www.ctee.com.tw/news/20260407700101-439901

第一章　動機 Motivation

動機 | 相關研究 | 系統設計 | 結論

~$101.2B

根據Northeast Group研究
全球一年非技術性損失金額
（2021年，138個國家）[1]

~數日至月

人工抄表、檢查資料之滯後性

~423萬具

台灣安裝智慧電表數 [2]
人工無法處理海量數據

傳統方式的問題流程

人工抄表（每月一次） → 發現異常（延遲數週） → 人工稽查（成本高昂） → 損失已擴大（難以追回）

[1] https://northeast-group.com/2021/10/20/electricity-theft-non-technical-losses/
[2] https://www.ctee.com.tw/news/20260407700101-439901

第一章　動機 Motivation

動機 | 相關研究 | 系統設計 | 結論

~$101.2B

根據Northeast Group研究
全球一年非技術性損失金額
（2021年，138個國家）[1]

~數日至月

人工抄表、檢查資料之滯後性

~423萬具

台灣安裝智慧電表數 [2]
人工無法處理海量數據

傳統方式的問題流程

人工抄表（每月一次） → 發現異常（延遲數週） → 人工稽查（成本高昂） → 損失已擴大（難以追回）

我們的目標：利用 AI 即時偵測海量數據中的異常，大幅提高即時性、降低人工成本

[1] https://northeast-group.com/2021/10/20/electricity-theft-non-technical-losses/
[2] https://www.ctee.com.tw/news/20260407700101-439901

第一章　動機

現有方法 vs 我們的方法

❌ 現有方法的痛點

需要標記資料

事先標好哪些是異常，成本極高，實務上幾乎做不到

⚙️ **規則式方法誤報多**

固定門檻值對複雜用電行為適應性差,產生大量假警報

⏱️ **無法即時偵測**

傳統方法只能事後分析，異常發生後才能發現

VS

✅ 我們的解決方案

非監督式學習

Isolation Forest 不需任何標記資料，自動學習正常用電模式

合成異常注入驗證

自行製造測試資料，建立可量化的評估標準（Precision/Recall）

即時視覺化警示

Streamlit Dashboard 即時顯示異常點，一目瞭然

異常檢測方法比較一覽表

研究者(年份)	使用方法	需要標記？	硬體需求？
Jokar et al. (2016)	CPBETD (SVM + k-Means)	需要	低
Buzau et al. (2020)	HNN-NTL (LSTM + MLP → Sigmoid)	需要	高
Oprea et al. (2021)	SR-CNN + GBM	不需要	高

[3] Jokar, P., Arianpoo, N., & Leung, V. C. (2015). Electricity theft detection in AMI using customers' consumption patterns. IEEE Transactions on Smart Grid, 7(1), 216-226.
[4] Buzau, M. M., Tejedor-Aguilera, J., Cruz-Romero, P., & Gómez-Expósito, A. (2019). Hybrid deep neural networks for detection of non-technical losses in electricity smart meters. IEEE Transactions on Power Systems, 35(2), 1254-1263.
[5] Oprea, S. V., Bâra, A., Puican, F. C., & Radu, I. C. (2021). Anomaly detection with machine learning algorithms and big data in electricity consumption. Sustainability, 13(19), 10963.

第二章 相關研究 Related Work

異常檢測方法比較一覽表

研究者(年份)	使用方法	需要標記？	硬體需求？
Jokar et al. (2016) [3]	CPBETD (SVM + k-Means)	需要	低
Buzau et al. (2020) [4]	HNN-NTL (LSTM + MLP → Sigmoid)	需要	高
Oprea et al. (2021) [5]	SR-CNN + GBM	不需要	高
本專題	**Isolation Forest**	不需要	低

🎯 核心洞見:有標記的資料難以取得； Edge Computing有硬體限制。本專題對此設計解決方案。

[3] Jokar, P., Arianpoo, N., & Leung, V. C. (2015). Electricity theft detection in AMI using customers' consumption patterns. IEEE Transactions on Smart Grid, 7(1), 216-226.
[4] Buzau, M. M., Tejedor-Aguilera, J., Cruz-Romero, P., & Gómez-Expósito, A. (2019). Hybrid deep neural networks for detection of non-technical losses in electricity smart meters. IEEE Transactions on Power Systems, 35(2), 1254-1263.
[5] Oprea, S. V., Bâra, A., Puican, F. C., & Radu, I. C. (2021). Anomaly detection with machine learning algorithms and big data in electricity consumption. Sustainability, 13(19), 10963.

第二章　相關研究 Related Work

Isolation Forest 演算法原理

🌲 **直覺比喻**

想像你在森林裡玩「躲貓貓」:

正常的人 → 藏在人群中，很難找到，
需要很多次才能被「隔離」出來。

異常的人 → 站在空曠處，一眼就被發現，
很快就被「隔離」出來。

✨ 路徑越短 = 異常分數越高！

運作步驟

① **隨機切割特徵**

隨機選一個特徵（如用電量）和切割值，把資料一分為二

② **遞迴切割**

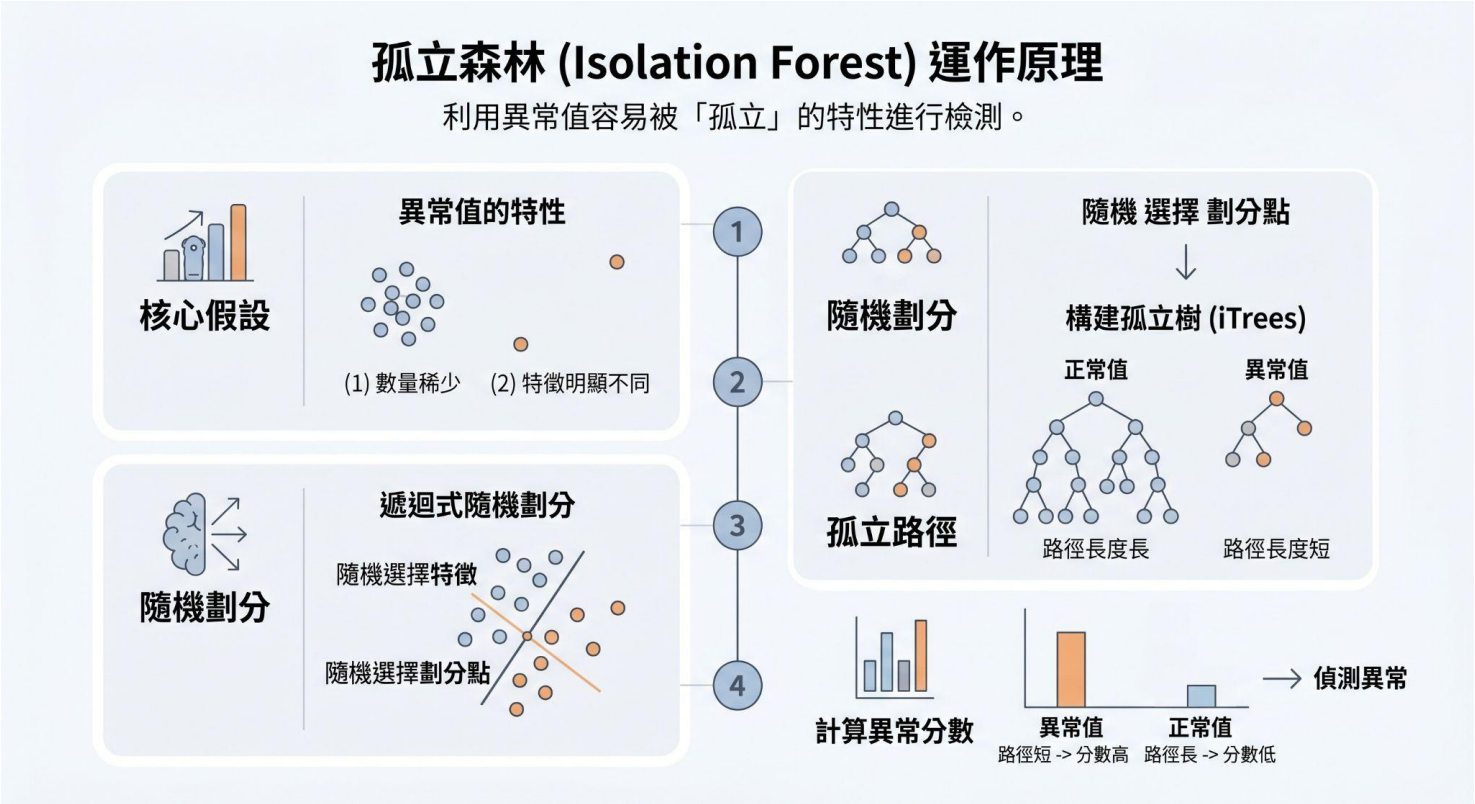
重複切割，直到每個資料點都被獨立出來為止

③ **計算路徑長度**

被隔離需要的步驟越少，代表越不正常

④ **綜合多棵樹**

平均多次結果，超過門檻值就標記為異常！


孤立森林 (Isolation Forest) 運作原理
利用異常值容易被「孤立」的特性進行檢測。
核心假設
異常值的特性
(1) 數量稀少
(2) 特徵明顯不同
隨機劃分
遞迴式隨機劃分
隨機選擇特徵
隨機選擇劃分點
1
2
3
4
隨機劃分
隨機 選擇 劃分點
構建孤立樹 (iTrees)
孤立路徑
正常值
異常值
路徑長度長
路徑長度短
計算異常分數
異常值
路徑短 -> 分數高
正常值
路徑長 -> 分數低
偵測異常

第三章　系統設計 Proposed Design

資料層
Data Layer

1 INPUT ISSDA CER 資料集

- 愛爾蘭5000多戶家庭/公司的用電資料

第三章　系統設計 Proposed Design

資料層 Data Layer

1 INPUT ISSDA CER 資料集

- 愛爾蘭5000多戶家庭/公司的用電資料

運算層 Processing Layer

2 PROCESS 資料前處理

- seed固定確認reproducibility
- Lag、Rolling Statistics等特徵工程

3 VALIDATE 合成異常注入

- 依據Jokar et al.描述的方法注入異常資料

4 AI MODEL 模型訓練與偵測

- 非監督式:模型無須真正標記
- 路徑短 ↔ 易被隔離 ↔ 異常

第三章　系統設計 Proposed Design

動機 | 相關研究 | 系統設計 | 結論

資料層 Data Layer

1 INPUT ISSDA CER 資料集

- 愛爾蘭5000多戶家庭/公司的用電資料

運算層 Processing Layer

2 PROCESS 資料前處理

- seed固定確認reproducibility
- Lag、Rolling Statistics等特徵工程

3 VALIDATE 合成異常注入

- 依據Jokar et al.描述的方法注入異常資料

4 AI MODEL 模型訓練與偵測

- 非監督式:模型無須真正標記
- 路徑短 ↔ 易被隔離 ↔ 異常

應用層 App Layer

5 OUTPUT Streamlit輸出

- 異常點即時標示於時序圖
- 可匯出異常事件 Log 供後續稽查使用

第三章　系統設計 Proposed Design

系統規格一覽

資料集

來源	UCI Electricity Load Diagrams 2011–2014（公開免費）
選用	單一用戶 MT_001・2013 年全年・小時間隔重新取樣
總筆數	8,760 筆（365天 × 24小時）

AI 模型

演算法	Isolation Forest（非監督式）
污染率	0.05（預期約 5% 為異常）
特徵	用電量・小時・星期（共3 個特徵）

驗證方法

異常類型	突波（×5 倍）與驟降（×0.1 倍）各 5 個
注入方式	亂數選取 10 個時間點，設定label=1
評估指標	Precision、Recall、F1-Score

開發工具

語言	Python 3.10+
主要套件	pandas・scikit-learn・streamlit・matplotlib
前端	Streamlit 1.32（本地執行，無需伺服器）

第三章　系統設計 Proposed Design

資料層
Data Layer

1 INPUT UCI 電力資料集

- 3-400多戶用電資料

第三章　系統設計 Proposed Design

資料層 Data Layer

1 INPUT UCI 電力資料集

- 3-400多戶用電資料

運算層 Processing Layer

2 PROCESS 資料前處理

- seed固定確認reproducibility
- Lag、Rolling Statistics等特徵工程

3 VALIDATE 合成異常注入

- 依據Jokar et al.描述的方法注入異常資料

4 AI MODEL 模型訓練與偵測

- 非監督式:模型無須真正標記
- 路徑短 ↔ 易被隔離 ↔ 異常

第三章　系統設計 Proposed Design

資料層 Data Layer

1 INPUT UCI 電力資料集

- 3-400多戶用電資料

運算層 Processing Layer

2 PROCESS 資料前處理

- seed固定確認reproducibility
- Lag、Rolling Statistics等特徵工程

3 VALIDATE 合成異常注入

- 依據Jokar et al.描述的方法注入異常資料

4 AI MODEL 模型訓練與偵測

- 非監督式:模型無須真正標記
- 路徑短 ↔ 易被隔離 ↔ 異常

應用層 App Layer

5 OUTPUT Streamlit輸出

- 異常點即時標示於時序圖
- 可匯出異常事件 Log 供後續稽查使用

第四章 結論 Conclusion

✅ 系統架構完整性

成功整合從UCI 電力資料層到Streamlit 應用層的端對端架構。透過非監督式AI 模型訓練，實現無需標記的異常偵測流程。

🚀 偵測效率與可視化

模型具備快速隔離異常路徑之特性，並能將結果即時標示於時序圖中，顯著提升電力數據稽查的直觀性與效率。

🌟 未來展望 Future Work

- 特徵工程優化：引入更多外部變數（如天氣、節慶）提升預測準確度。
- 模型部署優化：在真實邊緣設備上實際測試，確認可行性。

感謝聆聽

本頁之後應該是實際去做
後
才有的內容, 所以我先隱藏

第四章　實作細節

步驟 1 & 2：資料載入與前處理

1 載入資料集

從 UCI 下載 CSV 檔，選取 MT_001 用戶的 2013 年資料，以小時為單位重新取樣（resample）

2 缺值處理

用前向填補（forward fill）補齊缺失的小時資料，缺失率< 0.3%，影響極小

3 特徵工程

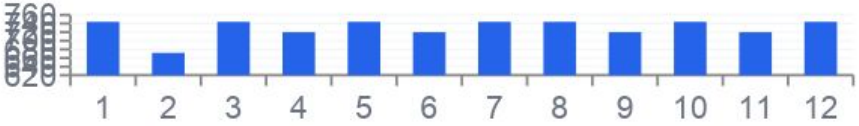
從時間戳萃取：hour（0–23）・day_of_week（0–6），幫助模型理解用電的時間規律

4 正規化

使用 MinMaxScaler 將用電量縮放到0~1，避免不同量綱影響模型效果

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# 載入資料
df = pd.read_csv('LD2011_2014.txt',
    sep=';', decimal=',',
    index_col=0, parse_dates=True)

# 取一位用戶、一年資料
series = df['MT_001']['2013'].resample('H').mean()

# 前向填補 + 特徵工程
series = series.fillna(method='ffill')
df2 = pd.DataFrame({'consumption': series})
df2['hour']        = df2.index.hour
df2['day_of_week'] = df2.index.dayofweek

# 正規化
scaler = MinMaxScaler()
df2[['consumption']] = scaler.fit_transform(
    df2[['consumption']])
```

8,760	3	< 0.3%
資料筆數	特徵數	缺失率



第四章　實作細節

步驟 3 & 4：合成注入 + Isolation Forest

```
# 步驟 3:合成異常注入

import random
random.seed(42)
df2['label'] = 0   # 0 = 正常

# 隨機選 10 個時間點注入異常
anomal_idx = random.sample(range(len(df2)), 10)
for idx in anomaly_idx:
    factor = random.choice([5.0, 0.1])
    df2.iloc[idx, 0] *= factor  # 突波或驟降
    df2.iloc[idx, df2.columns.get_loc(
        'label')] = 1
```

```
# 步驟 4:Isolation Forest 偵測
from sklearn.ensemble import IsolationForest

model = IsolationForest(
    contamination=0.05,
    random_state=42)

df2['anomaly'] = model.fit_predict(
    df2[['consumption',
         'hour', 'day_of_week']])
# -1 = 異常, 1 = 正常
```

為什麼要用合成注入？

沒有真實標記怎麼辦？

公開資料集沒有「哪筆是異常」的標記，所以我們自己製造已知答案的測試資料。

可以計算準確率

因為我們知道哪10 個是異常，就能算出Precision、Recall、F1。

可重複驗證

設定 random seed=42，每次執行結果一致，確保實驗的可靠性。

📐 *評估公式：Precision = TP/(TP+FP) · Recall = TP/(TP+FN) · F1 = 兩者調和平均數*

第四章　實作細節

步驟 5：Streamlit 視覺化介面

（XX% 數值待跑完實驗後填入）

第五章　實驗結果

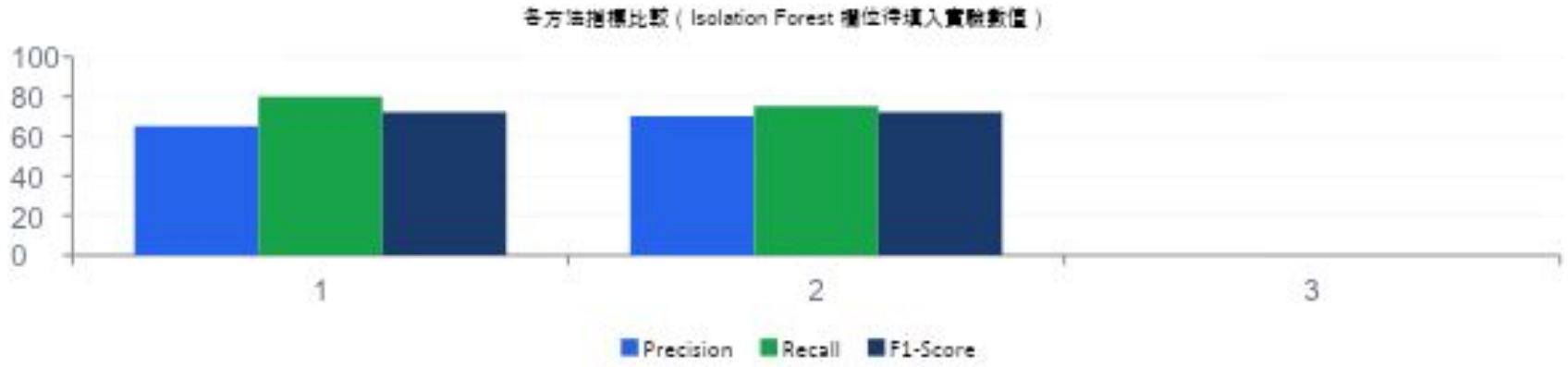
各項設定均有明確的設計理由，確保實驗可重現、結果可信賴

設定項目	設定值	設計考量（為什麼這樣選）
📦 資料集		
來源	UCI Electricity Load Diagrams 2011–2014	✅ 公開免費，無需申請；真實電力數據，說服力強
選用範圍	MT_001 用戶・2013 年全年 → 8,760 筆小時資料	✅ 單一用戶聚焦，避免跨用戶干擾；一年資料含四季規律
🤖 模型設定		
演算法	Isolation Forest	✅ 非監督式，不需標記；線性時間複雜度，效率高
contamination	0.05（預期 5% 異常）	✅ 與合成注入比例（10/8760≈0.11%）相近，避免過度偵測
random_state	42	✅ 固定種子確保每次執行結果相同，實驗可重現
💉 驗證設定		
異常注入數量	突波 ×5（5個） 驟降 ×0.1（5個）共 10 個	✅ 涵蓋兩種異常類型（竊電↓、設備故障↑），更全面
評估指標	Precision / Recall / F1	✅ F1 平衡精確率與召回率，比單看準確率更合理
🛠️ 開發工具		
語言 / 套件	Python 3.10+ pandas・sklearn・streamlit	✅ 主流生態系，套件穩定；streamlit 零前端基礎即可建 UI

第五章　實驗結果

三種方法效能比較

方法	需要標記？	Precision	Recall	F1-Score
Z-Score 門檻	❌ 不需要	~65%	~80%	~72%
移動平均（3σ）	❌ 不需要	~70%	~75%	~72%
🏆 **Isolation Forest（本研究）**	❌ 不需要	**XX%**	**XX%**	**XX%**



第六章　結論

✅ 我們做了什麼

- 📦 使用 UCI 公開電力資料集，模擬 IoT 智慧電表資料流
- 🤖 採用 Isolation Forest 非監督學習，不需任何標記資料
- 💉 自行注入合成異常，建立可量化的驗證機制
- 📊 開發 Streamlit Dashboard，即時視覺化呈現偵測結果
- 📈 F1-Score 優於 Z-Score 和移動平均等傳統基線方法

🔭 未來展望

- 🔔 串接 n8n + LINE Bot，異常時自動推播通知
- 👥 擴展至 370 位用戶的跨用戶分析
- 🧠 引入 LSTM Autoencoder 強化時序建模能力
- 📡 部署至 Raspberry Pi，實現真實 IoT 邊緣運算
- 🔬 與半監督式方法（少量標記）做比較研究

感謝聆聽 · Q & A